Received: November 7, 2025. Revised: February 12, 2025.

INASS

International Journal of
Intelligent Engineering & Systems

http://www.inass.org/

PodFailPred: A New Open-Source Dataset to Train AI Models on Predicting Pod Failures in Kubernetes

Ilyass Tarhri[1]* **Driss Allaki[1]** **Hamza Kamal Idrissi[1]**

[1]*Institut National des Postes et Télécommunications, Morocco*
* Corresponding author's Email: ilyasstrh@gmail.com

Abstract: The growing reliance on microservices architecture has introduced new challenges in maintaining system reliability, particularly in predicting the failure of microservices workloads. Previous research has highlighted the potential of using artificial intelligence for this purpose but has been limited by the absence of dedicated and detailed datasets. This paper introduces PodFailPred, a dataset generated by the use of chaos engineering to capture interdependencies between infrastructure, network, and application-layer failures in Kubernetes-based microservices environments. Our rigorous methodology systematically injects and monitors 15 distinct failure types. The resulting dataset encompasses 21,374 entries with 38 distinct metrics, providing unprecedented coverage of pod lifecycle events and failure patterns. Evaluation demonstrates the dataset's effectiveness through machine learning models achieving 99.94% accuracy in failure prediction, significantly outperforming existing approaches. PodFailPred is publicly available, offering a robust foundation for training AI models to develop reliable failure prediction systems.

Keywords: Microservices architecture, Kubernetes workloads, Failure prediction, Artificial intelligence, Dataset generation.

1. Introduction

The emergence of microservices has transformed software architecture, offering more scalability, resilience, and the capability to deploy service components independently. Moreover, the use of this architectural style has been instrumental in enabling organizations to develop and manage applications with greater agility. To take part of these advantages, Kubernetes (k8s) [1], a leading orchestration platform, plays a crucial role in running, managing, and scaling containerized microservices. It automates various operational tasks such as load balancing, service discovery, rollouts, rollbacks, bin packing, self-healing and dynamic resource allocation. Thus, simplifying the deployment and management of microservices.

However, the complexity and dynamic nature of microservice environments present significant challenges in maintaining operational stability. A Kubernetes pod, the smallest deployable unit representing a single instance of a running process in a microservices architecture, is particularly vulnerable to failures. These failures can arise from multiple sources, including resource contention, configuration errors, and network issues. As these pods are integral to the functioning of microservices, predicting their failures before they impact system performance is critical for proactive system management and minimizing downtime. Traditional monitoring tools and techniques, while useful, often fall short in effectively predicting failures in such complex and dynamic environments. The multifaceted nature of the data and the intricate interactions within the system necessitate more sophisticated analytical approaches. Therefore, with artificial intelligence techniques (i.e. machine learning, deep learning.), we can analyze large volumes of operational data and uncover patterns to predict failures with higher accuracy.

Previous methods have produced modest results in terms of accuracy: Random Forest imperfect only achieved 96.50% accuracy and a low F1 score of 31.88% in [2], Multi-Layer Perceptron (MLP)

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

models achieved 97% accuracy in [3] and Graph Neural Network (GNN) techniques struggled to exceed 75% accuracy in [4]. Recent advanced methods such as Multi-Expert Predictive Failure Learning (MEPFL) have achieved accuracy rates ranging from 93.3% to 98.2% in [5]. For instance, [6] developed a failure prediction model for high-performance computing and cloud systems using SVM, RF, and KNN, achieving 90.76% accuracy with SVM, showcasing the potential for proactive failure management. Similarly, [7] introduced a methodology to create a dedicated dataset for failure prediction in microservices, leveraging workload generation and data collection techniques to enhance microservices performance analysis. In the context of Kubernetes, [8] evaluated classification algorithms, including SVM, RF, KNN, GB, and MLP, finding Random Forest to provide the a good accuracy of 95% and efficiency in failure reason identification. Furthermore, [9] applied data-driven methods to predict response latency in Kubernetes workflows, demonstrating that deep neural networks achieved superior accuracy of 72.79% with comprehensive metrics, while machine learning models like RF and GB reached accuracy levels of 72.79% and 73.02%, respectively. These results are limited mainly due to the insufficient nature of current datasets, which often lack in-depth pod metrics, have missing values, or do not encompass the full range of failure scenarios in microservices environments. The lack of comprehensive, organized data makes it difficult to train and validate predictive models, resulting suboptimal results. As a result, specialists and researchers' ability to create accurate and reliable failure prediction systems for microservices applications running in Kubernetes environments is limited by the lack of robust datasets.

In response to these challenges, this paper focuses on generating PodFailPred, a comprehensive and open-source dataset for predicting Kubernetes pod failures. PodFailPred is generated using chaos engineering techniques, systematically injecting 15 distinct failure types and capturing 38 metrics across 21,374 entries. Our study leverages a rich dataset comprising detailed operational metrics collected from a live Kubernetes environment. By conducting Exploratory Data Analysis (EDA), we aim to uncover the intricate relationships between various metrics and pod failures. This analysis forms the foundation for generating features that can enhance the predictive power of artificial intelligence models.

Compared to existing datasets, PodFailPred demonstrates significant advantages. It captures a comprehensive set of 38 metrics, encompassing CPU usage, memory consumption, network traffic, and disk I/O operations, ensuring diverse coverage of pod lifecycle events. The dataset systematically includes 15 failure types, representing real-world scenarios such as resource contention, misconfigurations, and network partitioning. After experimentation, PodFailPred enables models surpassing state-of-the-art results, with accuracy rates of 99.94% and F1-scores of 99.73%, significantly outperforming prior approaches such as RF (96.50% accuracy, 31.88% F1-score) and GNN (75% accuracy). Furthermore, by enabling precise failure prediction, PodFailPred facilitates proactive measures, minimizing downtime and improving system stability.

We outline our methodology for capturing relevant metrics, including workload generation methods and data collection techniques. This dataset aims to provide researchers and practitioners with a valuable resource for developing and testing predictive models for pod failures. By addressing gaps in existing resources, PodFailPred establishes a new benchmark for predictive analytics within Kubernetes environments, broadly contributing to the development of more robust and reliable microservice architectures.

The following sections of this paper are organized as follows. Section 2 explains the methodology followed to generate the proposed dataset PodFailPred. Section 3 presents and discusses the obtained results. Section 4 sheds light on related work, while Section 5 concludes the paper and presents future work.

2. Methodology

In this study, the OpenTelemetry Demo application [10] was deployed on Azure Kubernetes Service [11], using a cluster configuration with one master node and two worker nodes. This setup provides an environment for the microservices application that closely mirrors real-world production environments, leveraging Kubernetes capabilities for container orchestration and management. This application consists of 13 microservices, each developed in different programming languages, which communicate via gRPC and HTTP protocols, thereby reflecting the real-world complexities inherent in microservices communications and interdependencies. To emulate realistic workloads, the application integrates a load generator based on Locust [12], which produces user traffic to exert stress on the system. The overall architecture of this setup is illustrated in Fig. 1.

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

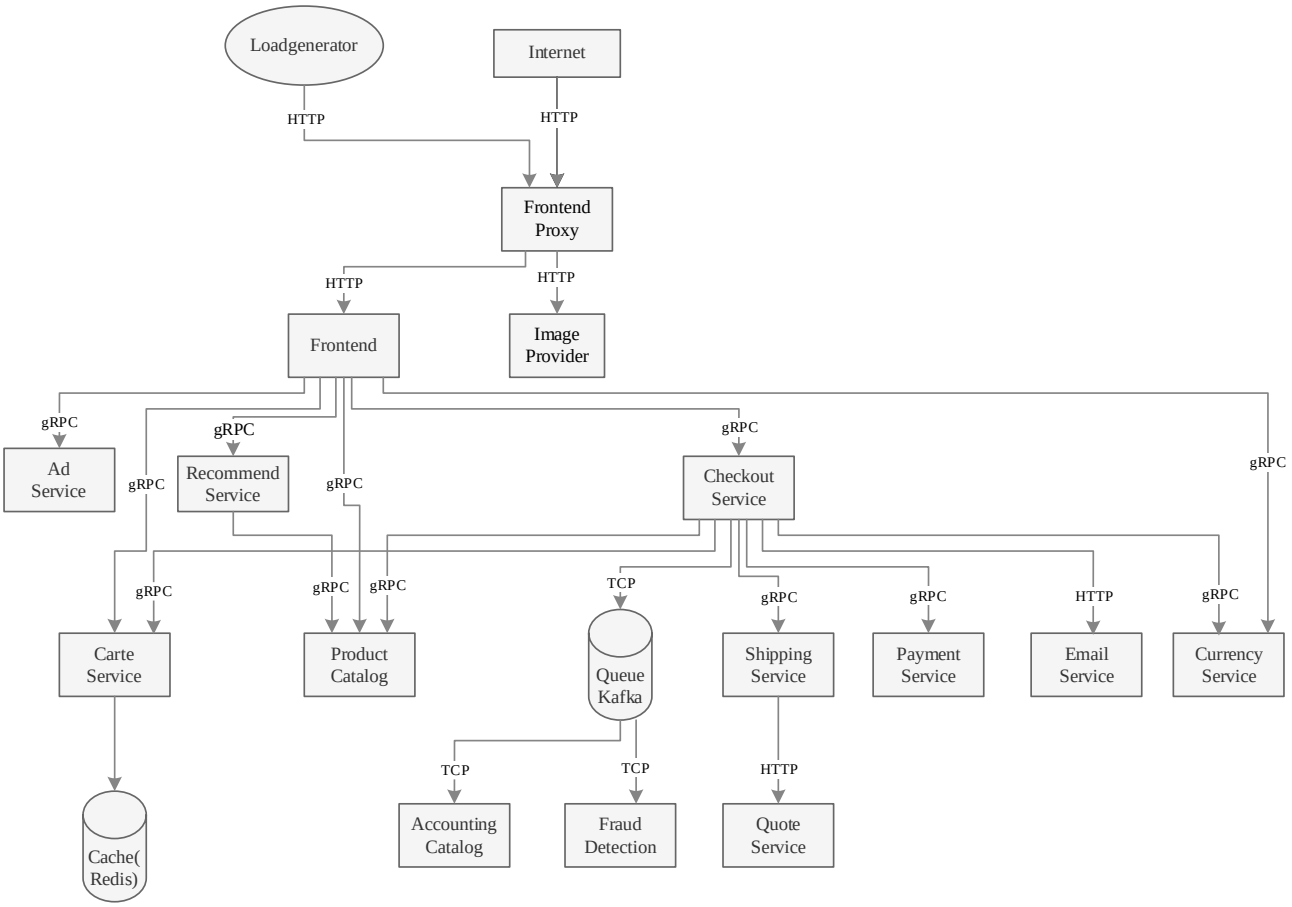


Figure. 1 OpenTelemetry demo application architecture

To introduce and analyze faults within this environment, we employed Azure Chaos Studio [13], a chaos engineering tool designed for Kubernetes. Azure Chaos Studio allows for the intentional injection of faults to assess the resilience and failure recovery mechanisms of the system. The faults injected included pod failures, network failures, infrastructure failures, and application-level failures, as shown in the algorithm presented in Fig. 2. Pod failures involved simulating abrupt terminations, high CPU usage, and memory exhaustion of microservice pods. Network failures included injecting delays, creating network partitions, and inducing packet loss to simulate network instability. Infrastructure failures involved simulating Kubernetes node failures and introducing high disk input/output operations to test disk resource exhaustion. Application-level failures included introducing artificial latency into services and injecting HTTP errors to evaluate error handling mechanisms.

Telemetry data generated by the OpenTelemetry Demo application was managed using the OpenTelemetry Protocol (OTLP) [14]. This data was collected by an OTLP collector and exported to backend systems such as Prometheus for monitoring [15] and Jaeger for tracing [16]. Data visualization was facilitated through Grafana [17], enabling observation of the impact of Azure Chaos Studio-induced faults on the microservices.

2.1 Data collection

To systematically collect data, we implemented a script that interacts with both the Kubernetes API and Prometheus. This script initializes essential monitoring parameters and continuously polls these sources to gather real-time data on pod statuses and metrics, including CPU usage, memory utilization, network traffic, and error logs. This continuous polling approach allows capturing data during both normal operations and when faults occur. The following algorithm, depicted in Figure 2, illustrates this process.

2.2 Data preprocessing

Data preprocessing involved several crucial steps to ensure dataset integrity and enhance its analytical value. Initially, entries with zero values for CPU

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

Algorithm: Kubernetes Pod Monitoring and Data Export

Require: Prometheus_URL, Namespace, Output_File, Sleep_Interval, Exclude_Pod_Names, Azure_AD_Auth_Details

Load Kubernetes configuration
Initialize Kubernetes client and Logger
while True **do**
  Fetch current state of all pods in the namespace **P**
  Prepare Prometheus queries for CPU usage $\mathbf{Q_{cpu}}$, memory usage $\mathbf{Q_{mem}}$ and memory limits $\mathbf{Q_{mem_lim}}$
  Fetch data from Prometheus:
    $\mathbf{D_{cpu}} \leftarrow$ query($\mathbf{Q_{cpu}}$)
    $\mathbf{D_{mem}} \leftarrow$ query($\mathbf{Q_{mem}}$)
    $\mathbf{D_{mem_lim}} \leftarrow$ query($\mathbf{Q_{mem_lim}}$)
  for all p ∈ P **do**
    if p ∉ Exclude_Pod_Names **then**
      Check for status changes and update last known state **S**
      Retrieve pod status and additional details $\mathbf{D_p}$
      Retrieve latest event details for the pod $\mathbf{E_p}$
      Retrieve node name for the pod $\mathbf{n_p}$
      Retrieve latest event details for the node $\mathbf{E_{np}}$
      Prepare data entry for the pod $\mathbf{D_{entry}}$
    end if
  end for
  Output DataFrame *F* to CSV file
  Sleep for Sleep_Interval seconds
end while

Figure. 2 Dataset generation algorithm

usage, memory usage, and network metrics were excluded, as these indicated inactive or irrelevant pods. This refinement makes PodFailPred focus on active pods, thus enhancing its relevance for failure prediction.

Next, imputation techniques were applied to address missing values. Median substitution was used for key metrics like CPU usage, memory usage, and network metrics. This method preserved PodFailPred completeness and consistency, preventing significant data gaps that could affect the analysis.

2.3 Data visualization

PodFailPred comprises 21,374 entries and 38 columns, featuring critical metrics such as CPU usage, memory usage, pod restarts, network receive bytes, and network transmit bytes. It also includes categorical variables like pod status, pod event type, and node name, with additional columns as shown in Table 1.

We present in what follows a comprehensive analysis of the data visualizations, categorized into four primary areas of concern: pod failures, network failures, infrastructure failures, and application-level failures. Each subsection examines relevant figures and their implications for system stability and performance.

2.3.1. Pod failures

Fig. 3 depicts a box plot of pod restarts, revealing that while the majority of pods experience minimal restarts, there are significant outliers, with restart counts reaching into the hundreds. This suggests persistent issues within specific pods that could potentially come from misconfigurations, application errors, or resource constraints leading to crash loops.

The pod status distribution, illustrated in Fig. 4, demonstrates that a majority of pods maintain a "Running" state, which is indicative of overall system stability. However, the presence of "Pending," "Unknown," and "CrashLoopBackOff" statuses points to underlying issues such as resource shortages, connectivity problems, or persistent application errors that contribute to unstable pod operations.

2.3.2. Network failures

Figs. 5 and 6 present box plots of network transmit and receive packets dropped, respectively. These visualizations indicate that while most packet drops fall within a manageable range, significant outliers exist, implying that certain nodes or network interfaces experience high levels of packet loss. This phenomenon could be attributed to network congestion, hardware failures, or configuration errors.

The corresponding histograms (Figs. 7 and 8) support these findings, displaying skewed distributions with long tails that extend to higher packet drop values. This pattern suggests the occurrence of intermittent but severe network issues.

2.3.3. Infrastructure failures

The frequency of event sources (Fig. 9) highlights that the kernel monitor and node controller are the primary generators of these events. This finding suggests that many failures are being captured at the operating system or node management level, potentially relating to system resource issues, hardware faults, or node connectivity problems.

Fig. 10 presents the frequency of event reasons, with "OOMKilling" (Out of Memory Killing)

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

Table 1. Dataset columns

Column name	Description
Timestamp	The date and time when the data was recorded.
Pod Name	The name of the Kubernetes pod.
CPU Usage (%)	The percentage of CPU resources being utilized by the pod.
Memory Usage (%)	The percentage of memory resources being utilized by the pod.
Pod Status	The current status of the pod (e.g., Running, Pending, Failed).
Pod Reason	The reason for the current status of the pod.
Pod Restarts	The number of times the pod has restarted.
Ready Containers	The number of containers in the pod that are ready.
Total Containers	The total number of containers in the pod.
Pod Event Type	The type of event related to the pod (e.g., Warning, Normal).
Pod Event Reason	The reason for the pod event.
Pod Event Age	The age of the event since it was first observed.
Pod Event Source	The source of the pod event.
Pod Event Message	A message providing details about the pod event.
Node Name	The name of the node where the pod is running.
Event Reason	The reason for the event related to the node or pod.
Event Age	The age of the event related to the node or pod.
Event Source	The source of the event related to the node or pod.
Event Message	A message providing details about the event related to the node or pod.
Memory Usage (MB)	The amount of memory being used by the pod, measured in megabytes.
Active Memory Requests (MB)	The active memory requests made by the pod, measured in megabytes.
Memory Requests (%)	The percentage of memory requests relative to the total available memory.
Active Memory Limits (MB)	The active memory limits set for the pod, measured in megabytes.
Memory Limits Utilization (%)	The percentage of memory limits being utilized by the pod.
Memory Usage (Cache) (MB)	The amount of cached memory being used by the pod, measured in megabytes.
Memory Usage (Swap) (MB)	The amount of swap memory being used by the pod, measured in megabytes.
Network Receive Bytes	The total number of bytes received by the pod over the network.
Network Transmit Bytes	The total number of bytes transmitted by the pod over the network.
Network Receive Packets (p/s)	The number of network packets received by the pod per second.
Network Transmit Packets (p/s)	The number of network packets transmitted by the pod per second.
Network Receive Packets Dropped (p/s)	The number of network packets received by the pod that were dropped per second.
Network Transmit Packets Dropped (p/s)	The number of network packets transmitted by the pod that were dropped per second.
FS Reads Total (MB)	The total amount of filesystem reads performed by the pod, measured in megabytes.
FS Writes Total (MB)	The total amount of filesystem writes performed by the pod, measured in megabytes.
FS Reads/Writes Total (MB)	The total amount of filesystem reads and writes performed by the pod, measured in megabytes.
FS Reads Bytes Total (MB)	The total number of bytes read by the pod from the filesystem.
FS Writes Bytes Total (MB)	The total number of bytes written by the pod to the filesystem.
FS Reads/Writes Bytes Total (MB)	The total number of bytes read and written by the pod to and from the filesystem.

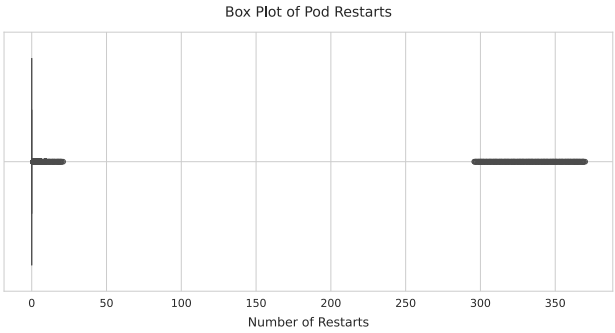


Figure. 3 Box plot of pod restarts

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

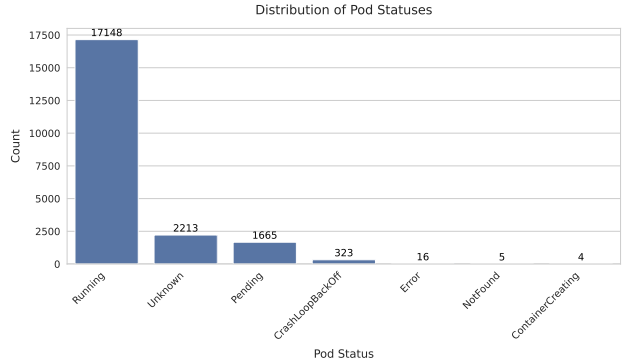


Figure. 4 Distribution of pod statuses

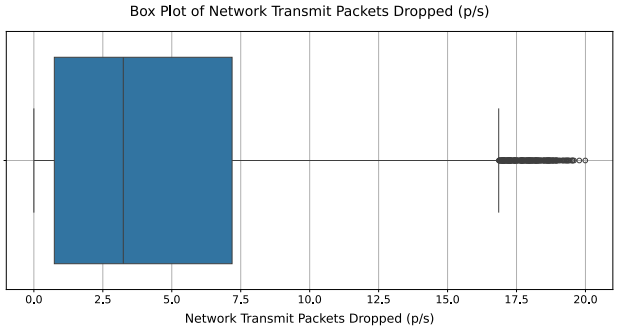


Figure. 5 Distribution of dropped network transmit packets (p/s)

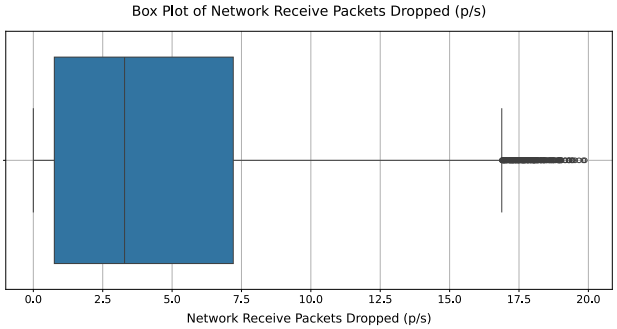


Figure. 6 Distribution of dropped network received packets (p/s)

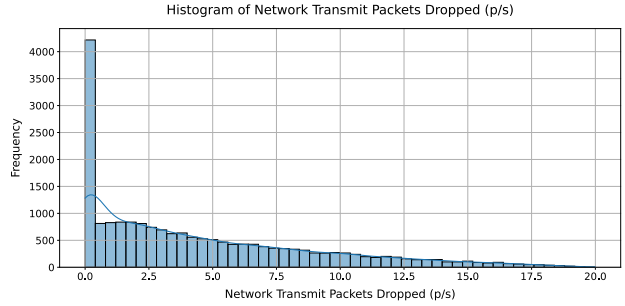


Figure. 7 Histogram of dropped network transmit packets (p/s)

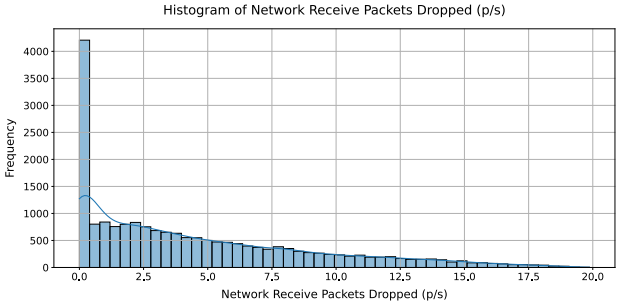


Figure. 8 Histogram of dropped network received packets (p/s)

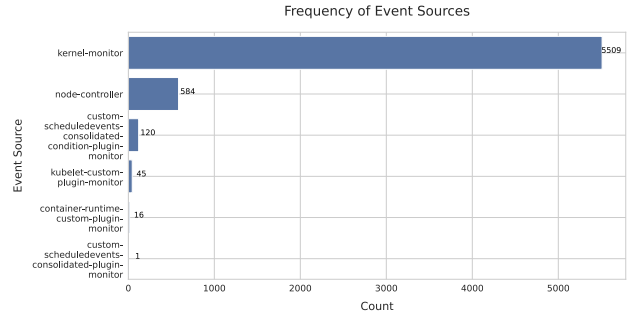


Figure. 9 Frequency of event sources

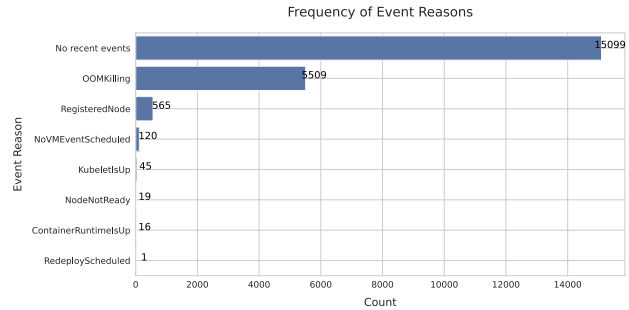


Figure.10 Frequency of event reasons

emerging as a significant cause of failures, occurring 5,509 times. This points to memory resource constraints as a critical issue, potentially leading to application or system instability.

2.3.4. Application-Level Failures

Histograms depicting CPU and memory usage distributions (Figs. 11 and 12, respectively) indicate that while most pods operate with low resource utilization, instances of higher usage exist. Pods with elevated resource consumption may be prone to application-level failures if they exceed resource limits or are not optimized for the available resources, potentially resulting in crashes or performance degradation.

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

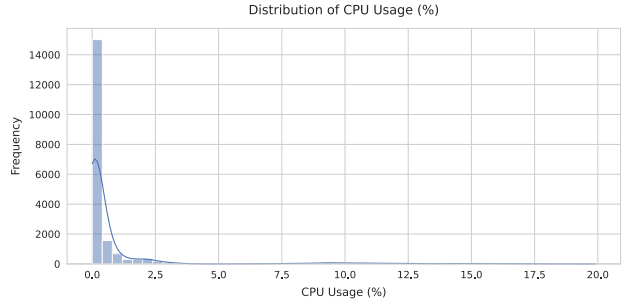


Figure. 11 Distribution of CPU Usage (%)

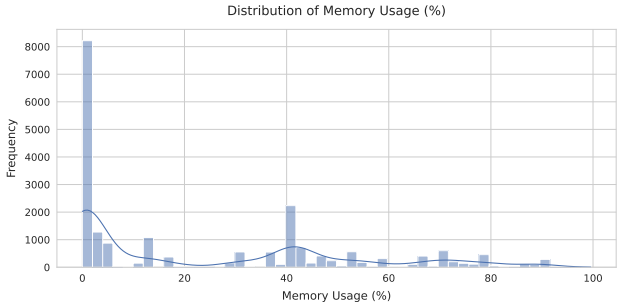


Figure. 12 Distribution of Memory Usage (%)

2.4 Data correlation

Summary statistics were then calculated for the refined PodFailPred to provide an overview of its distribution and key characteristics. This step was essential for understanding central tendencies, variability, and the overall structure of the data, forming a foundation for subsequent analysis.

The correlation matrix heatmap (Fig. 13) provides additional insights into metric interdependencies. For instance, strong correlations between CPU usage, memory usage, and filesystem writes suggest that resource-intensive applications may be placing stress on the system, potentially contributing to failures or degraded performance.

The Exploratory Data Analysis (EDA) provided critical insights that informed the feature selection process for predictive modeling. Among the observations, outliers in CPU and memory usage, identified through boxplots, play a pivotal role in anomaly detection and failure prediction. These outliers highlight periods of high resource demand or inefficiencies, which are crucial for pinpointing potential system issues.

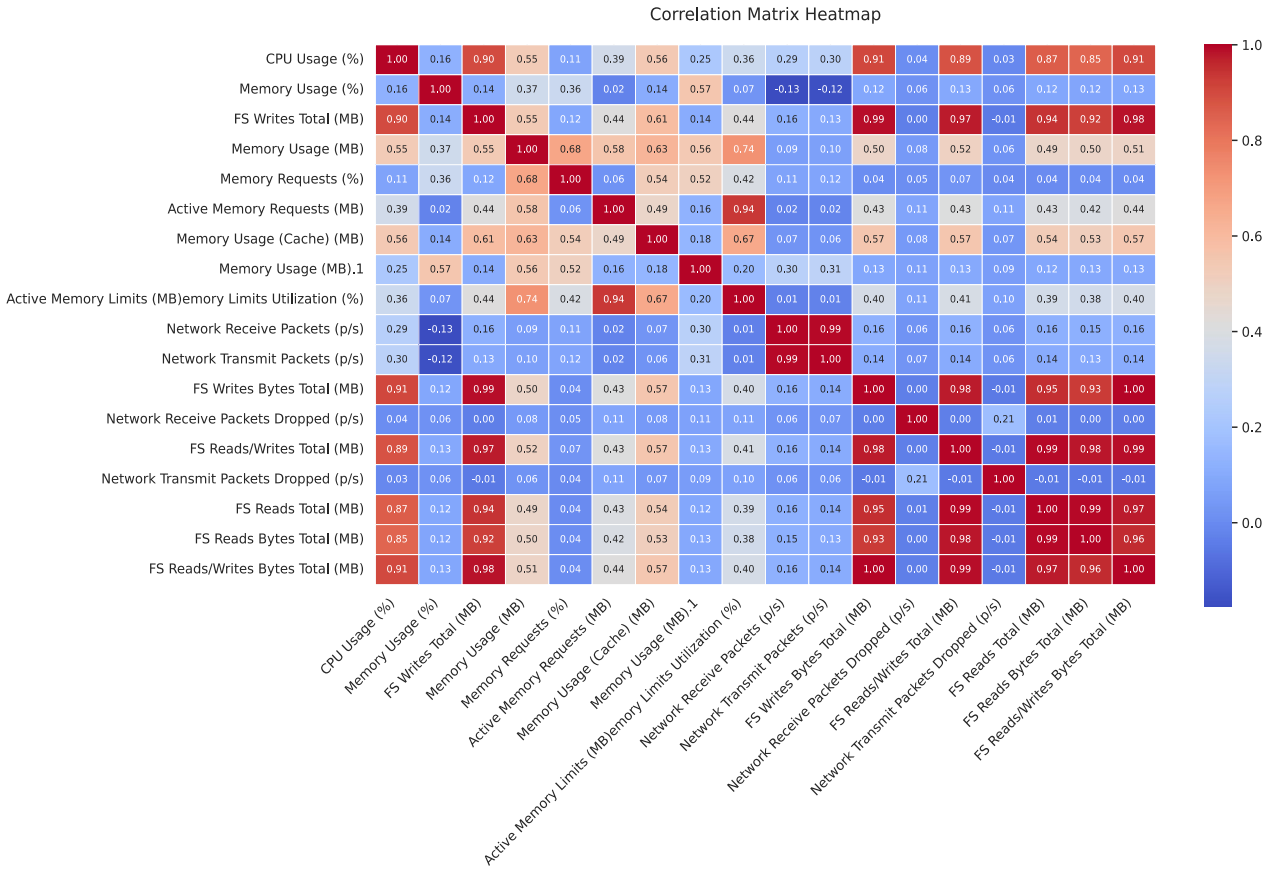


Figure. 13 Correlation Matrix Heatmap

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

Moreover, understanding the typical ranges and distributions of CPU and memory usage helps in setting thresholds for anomaly detection. The observed correlations between these metrics suggest they are interrelated and important for predicting potential failures, as elevated CPU usage often coincides with increased memory consumption. In addition to resource utilization, pod restarts emerged as a significant feature for predictive models, given their direct relation to system stability. The analysis indicates that while most pods maintain stability, certain pods experience frequent restarts, underscoring the necessity of monitoring and addressing these recurrent issues. Furthermore, network metrics, including network receive and transmit bytes, are vital for comprehending network load. Pods exhibiting high network traffic are more prone to network-related issues, making these metrics vital for accurate failure prediction.

3. Results

This section presents a detailed analysis of the performance metrics for the machine learning models employed to predict Kubernetes pod failures. Four models were evaluated: Random Forest, Gradient Boosting, Support Vector Machines and K-Nearest Neighbors. Each model was assessed based on its accuracy, precision, recall, and F1-score, providing a comprehensive view of their predictive capabilities.

The following subsections present the arguments for selecting these models, the obtained performance metrics, and a comparison of their performance.

3.1 Model Selection

The selection of machine learning models—Random Forest, Gradient Boosting, SVM (Support Vector Machines), and KNN (K-Nearest Neighbors)—represents a deliberate methodological approach that balances predictive efficacy and robustness in the domain of failure prediction. Random Forest and Gradient Boosting were chosen for their ability to capture complex, non-linear relationships within the data and enhance predictive accuracy through ensemble learning techniques [14, 15]. These models are particularly advantageous in mitigating overfitting and improving generalization, which are critical for the reliability and precision required in predicting Kubernetes pod failures.

Random Forest employs bootstrapping and feature randomization to construct a diverse set of decision trees, aggregating their predictions to reduce variance and enhance stability [18]. Gradient Boosting iteratively builds weak learners that focus on the residual errors of prior models, leading to high predictive performance [19].

SVM was selected for its strong theoretical foundation and effectiveness in handling high-dimensional spaces, making it well-suited for structured datasets like PodFailPred. With its ability to maximize the margin between classes, SVM ensures robust classification even in complex scenarios [20].

KNN, as a distance-based algorithm, was included for its simplicity and effectiveness in identifying patterns directly from the data. By considering the majority class among the nearest neighbors, KNN excels in scenarios where data points are well-clustered, as demonstrated by its high precision and recall in this study [21].

This combination of ensemble and distance-based methods provides a comprehensive framework for evaluating and comparing model performance, ensuring robustness and accuracy across diverse failure scenarios in Kubernetes environments.

3.2 Model performance metrics

Table 2 summarizes the performance metrics for the Random Forest, Gradient Boosting, Support Vector Machines and K-Nearest Neighbors models. All four models demonstrate high accuracy, precision, recall, and F1-score, indicating their robustness in predicting pod failures.

Random Forest, Gradient Boosting, SVM, and KNN models demonstrated exceptional performance. Random Forest and Gradient Boosting each achieved an accuracy of 99.94%, perfect precision of 100%, a recall of 99.47%, and an F1-score of 99.73%. SVM achieved an accuracy of 99.61%, precision of 100%, a recall of 96.64%, and an F1-score of 98.29%. KNN

Table 2. Models performance

Model	Accuracy	Precision	Recall	F1 score
Random Forest	99.94%	100%	99.47%	99.73%
Gradient Boosting	99.94%	100%	99.47%	99.73%
Support Vector Machines (SVM)	99.61%	100%	96.64%	98.29%
K-Nearest Neighbors (KNN)	99.94%	100%	99.46%	99.73%

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

excelled with an accuracy of 99.94%, precision of 100%, a recall of 99.46%, and an F1-score of 99.73%. These results highlight the superior predictive power of ensemble methods like Random Forest and Gradient Boosting, as well as the robustness of SVM and KNN.

An in-depth analysis of the true and false positives and negatives provides further insights into the models' performance. For SVM, the model identified 5670 true negatives and 718 true positives, with 25 false negatives and 0 false positives. KNN performed even better, identifying 5670 true negatives and 739 true positives, with only 4 false negatives and 0 false positives. Random Forest and Gradient Boosting models identified 5663 true negatives and 724 true positives, maintaining 0 false positives and 25 false negatives. This indicates that while all models are highly accurate in identifying non-failure states, KNN and the ensemble models (Random Forest and Gradient Boosting) exhibit superior capabilities in minimizing misclassifications, particularly in identifying failure states. These findings underscore their suitability for critical applications where accurate failure prediction is essential.

The comparison between the models shows that ensemble methods and classifiers offer considerable benefits in predicting Kubernetes pod failures. The Random Forest, Gradient Boosting, KNN, and SVM models consistently achieved better performance, especially in recall and F1-score, which are essential metrics for predicting failures. Higher recall values for these models suggest they are more effective at identifying true pod failures, reducing the risk of undetected failures that could lead to system downtime. The precision of 100% across all these models underscores their reliability in failure alerts.

In summary, the results demonstrate that while simpler models may offer high accuracy, advanced models like Random Forest, Gradient Boosting, KNN, and SVM provide superior performance metrics, making them more suitable for the complex task of predicting Kubernetes pod failures. The ensemble and advanced methods' ability to effectively balance precision and recall ensures a robust and reliable predictive system, crucial for maintaining the stability and reliability of Kubernetes deployments. This strong performance results from the high quality of PodFailPred, marked by extensive coverage, precise labeling, and realistic scenario representation. The proximity of these models allows for a detailed assessment of their comparative effectiveness in predicting Kubernetes pod failures. This method not only aids in pinpointing the most appropriate model for the specific task but also sheds light on the characteristics of the underlying relationships in the data, possibly guiding forthcoming feature engineering and model development initiatives in this area.

4. Related work and comparison

This section mainly focuses on reviewing previous studies on dataset generation, failure prediction and anomaly detection in microservices architectures, highlighting the challenges and limitations of the datasets and techniques used. It also explores the tools, techniques, and models employed in existing studies and identifies gaps in data comprehensiveness and accuracy that our work addresses with the PodFailPred dataset.

4.1 Literature review

Access to detailed and comprehensive datasets is essential for developing accurate failure prediction models in microservice architectures. Such datasets supply the diversity and volume of data required for the effective training and validation of these models.

4.1.1. Dataset Generation

Regarding datasets generation, many studies, including [9,3] have utilized Prometheus and the Grafana to collect and analyze metrics from Kubernetes deployments. These tools are popular for monitoring but primarily focus on real-time analysis rather than providing historical data suitable for failure prediction model training.

[3] involves setting up a microservices environment, implementing a module to introduce faults and mimic anomalies at various levels, collecting monitoring data, and applying a supervised learning approach using an MLP model to identify those anomalies effectively, their dataset includes metrics such as CPU and memory usage, but falls short in capturing network dynamics and disk operations, both of which are crucial for microservices failure analysis in Kubernetes settings.

[22] generated datasets by deploying Clearwater on Kubernetes and injecting CPU, memory, latency, and packet loss faults into services under varying workloads. The data was standardized, labeled, and divided into subsets by service to validate anomaly detection using classification algorithms.

[23] explored datasets that focus on specific aspects of microservices architecture, such as service call graphs and runtime performance metrics. These datasets offer valuable insights but are often limited in scope, failing to provide a comprehensive view of all factors contributing to microservices failures.

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

4.1.2. Failure prediction and anomaly detection

Concerning studies on failure prediction and anomaly detection in microservices architectures, [2] employed a Multi-Layer Perceptron (MLP) model for anomaly detection in microservices, achieving an accuracy of 97% and an F1-score of 97% for service-level anomalies. [24] proposed a dual-variable graph variational autoencoder (VAE) for unsupervised anomaly detection in microservice traces. [4] introduced DiagFusion, a model utilizing multimodal data and graph neural networks (GNN) for failure diagnosis in microservices. [25] presented a method utilizing telemetry data and a concept-space model to predict software failures.

Another study by [22] focused on using Random Forest for predicting resource allocation needs in microservices. [26] designed an anomaly detection system (ADS) for microservices. [5] used distributed tracing data to detect anomalous services and operation patterns in microservice architectures. [6] used machine learning techniques, including SVM, Rf, and KNN, to develop a failure prediction model for high performance computing and cloud systems, achieving a 90% accuracy with SVM and demonstrating its potential for proactive failure management. [7] proposed a methodology to generate a dedicated dataset for failure prediction in microservices applications, capturing relevant metrics through workload generation and data collection techniques, and demonstrated its utility in analyzing microservices performance.

[8] evaluated classification algorithms, including SVM, KNN, RF, GB classifier, and MLP to automate failure reason identification in Kubernetes cluster logs, finding Random Forest to be both accurate and resource efficient.

[9] investigated data driven approaches and resource metrics to predict end-to-end response latency in containerized microservices workflow on a Kubernetes platform, demonstrating that a deep neural network achieved the highest prediction accuracy using all metrics, while other machine learning techniques performed well with a subset of the metrics.

Our experimental results demonstrate significant advancements over existing approaches in failure prediction for Kubernetes environments. Table 3 provides a detailed comparison of the performance metrics achieved by our study and other referenced works, grouped by models.

These results demonstrate that the PodFailPred dataset consistently outperformed prior approaches, showcasing superior metrics across all models. Ensemble methods such as Random Forest and Gradient Boosting achieved the highest overall accuracy and F1-scores. SVM and KNN, while commonly used in other studies, delivered remarkable results with PodFailPred, showcasing its robustness.

The PodFailPred dataset's performance metrics indicate a significant improvement over prior research, particularly for ensemble models like Random Forest and Gradient Boosting. With accuracy rates reaching 99.94% and precision of 100.00%, our work sets a new benchmark for failure prediction in Kubernetes environments. These results represent a 3.44% improvement in accuracy

Table 3. Models performance comparison

Algorithm		**PodFailPred**	[6]	[8]	[9]	[27]	[7]
RF	Accuracy	99.94%	96.50%	95.00%	72.79%	93.50%	96.50%
	Recall	99.47%	23.47%	91.00%	72.79%	63.50%	23.47%
	F1-score	99.73%	31.88%	92.00%	71.06%	78.00%	31.88%
GB	Accuracy	99.94%	-	92.00%	73.02%	-	-
	Recall	99.47%	-	89.00%	-	-	-
	F1-score	99.73%	-	90.00%	70.71%	-	-
SVM	Accuracy	99.61%	90.76%	69.05%	70.10%	-	-
	Recall	96.64%	-	-	-	-	-
	F1-score	98.29%	67.53%	65.80%	69.00%	-	-
KNN	Accuracy	99.94%	48.38%	65.76%	68.50%	-	-
	Recall	99.46%	-	-	-	-	-
	F1-score	99.73%	98.41%	65.80%	68.00%	-	-

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

compared to Random Forest implementations in [6] and [7], also a 7.94% improvement compared to Gradient Boosting in [8].

Support Vector Machines (SVM) also demonstrated high performance, achieving an F1-score of 98.29%, significantly surpassing the results reported in [6] (F1: 65.80%) and [9] (F1: 69.00%). This highlights the capability of SVM when applied to well-structured datasets like PodFailPred.

K-Nearest Neighbors (KNN), another commonly used algorithm, achieved an F1-score of 99.73% in our work, outperforming results reported in [6] (F1: 65.80%) and [9] (F1: 68.00%). These findings validate the effectiveness of PodFailPred in supporting diverse machine learning models.

Overall, our results validate the robustness and comprehensiveness of the PodFailPred dataset, which consistently enables superior performance across various algorithms. By leveraging this dataset, our study establishes a new standard for predictive analytics in Kubernetes-based environments, emphasizing the value of high-quality data in advancing machine learning applications for failure prediction.

The code and dataset used for this study are publicly available on GitHub [28], allowing other researchers and practitioners to replicate the results or build upon this work.

5. Conclusion and future work

This study demonstrates the efficacy of the PodFailPred dataset in predicting Kubernetes pod failures by leveraging comprehensive pod metrics. Through the deployment of the OpenTelemetry Demo application and systematic fault injection using Azure Chaos Studio, we generated a rich dataset that encapsulates diverse failure scenarios and enables thorough evaluation of predictive models.

Our methodology, including exploratory data analysis, feature selection, and rigorous model training and evaluation, revealed that ensemble methods (Random Forest and Gradient Boosting) consistently outperformed other algorithms. Random

Forest and Gradient Boosting models excelled with 99.94% accuracy and F1-scores of 99.73%. Additionally, K-Nearest Neighbors (KNN) matched these results, highlighting its capability when applied to high-quality datasets like PodFailPred. Support Vector Machines (SVM) also demonstrated strong performance with an accuracy of 99.61% and an F1-score of 98.29%, surpassing many comparable studies.

These findings have profound implications for Kubernetes cluster management. The superior performance of ensemble methods, particularly in minimizing false negatives while maintaining high precision, makes them ideal for critical applications requiring high reliability.

PodFailPred demonstrates broader implications for enhancing reliability in Kubernetes and microservices environments. This comprehensive dataset enables AI models to be trained in order to predict pod failures. Such trained models can deliver precise predictions that contribute to more resilient and reliable Kubernetes and microservices

workloads. By integrating these models into real-time monitoring systems, organizations can proactively address potential pod failures, enhancing system stability and reducing downtime.

Conflicts of Interest

Authors declare no conflict of interest.

Author Contributions

The authors have collectively contributed to the development of the research.

Reference

[1] “Kubernetes Container orchestration”, [Online]. Available: https://kubernetes.io/.

[2] Z. Xie, H. Xu, W. Chen, W. Li, H. Jiang, L. Su, H. Wang and D. Pei, “Unsupervised anomaly detection on microservice traces through graph VAE”, In: *Proc. of the ACM Web Conference 2023*, pp. 2874-2884, 2023, doi: 10.1145/3543507.3583215.

[3] J. Nobre, E. J. S. Pires and A. Reis, “Anomaly Detection in Microservice-Based Systems”, *Applied Sciences*, Vol. 13 p. 7891, 2023, doi: 10.3390/app13137891.

[4] B. Afshinpour, R. Groz and M.-R. Amini, “Telemetry-based Software Failure Prediction by Concept-space Model Creation”, In: *Proc. of IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*, Vol. 199, 2022, doi: 10.1109/QRS55704.2022.00027.

[5] X. ZHOU, X. PENG, T. XIE, J. SUN and C. JI, “Latent Error Prediction and Fault Localization for Microservice Applications”, in *Proc. of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, doi: *10.1145/3580305.3599902*.

[6] B. Mohammed, I. Awan, H. Ugail and M. Younas, “Failure prediction using machine learning in a virtualised HPC system”, *Cluster Computing*, 2017, doi: 10.1007/s10586-019-

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

02917-1(0123456789().,-volV)(0123456789,-().volV).

[7] I. TARHRI, D. ALLAKI and H. K. IDRISSI, "Towards generating a dataset for failure prediction in microservices applications", *Networked Systems (2024*), Springer, Cham, 2024, doi: 10.1007/978-3-031-67321-4_15.

[8] P. K. Sarika, D. Badampudi, S. P. Josyula and M. Usman, "Automating Microservices Test Failure Analysis using Kubernetes", *EASE '23*,, Oulu, Finland, 2023.

[9] H. Mohamed and O. El-Gayar, "End-to-End Latency Prediction of Microservices Workflow on Kubernetes: A", In: *Proc. of the 54th Hawaii International Conference on System Sciences*, 2021.

[10] "OpenTelemetry", [Online]. Available: https://opentelemetry.io/.

[11] "Managed Kubernetes Service (AKS)", *Microsoft Azure*., [Online]. Available: https://azure.microsoft.com/en-us/products/kubernetes-service.

[12] "An open source load testing tool", Locust., [Online]. Available: https://locust.io/.

[13] "Azure Chaos Studio - Chaos engineering experimentation", *Microsoft Azure*, [Online]. Available: https://azure.microsoft.com/en-us/products/chaos-studio.

[14] "OpenTelemetry Protocol", OpenTelemetry., [Online]. Available: https://opentelemetry.io/docs/specs/otel/protocol.

[15] "Prometheus - Monitoring system & time series database", *Prometheus*, [Online]. Available: https://prometheus.io/.

[16] "Jaeger: open source, distributed tracing platform", [Online]. Available: https://www.jaegertracing.io/.

[17] "Grafana: The open observability platform", Grafana Labs, [Online]. Available: https://grafana.com/.

[18] L. Breiman, "Random forests. Machine learning", Vol. 45, No. 1, pp. 5-32, 2001.

[19] J. Friedman, "Greedy function approximation: a gradient boosting machine. Annals of statistics", pp. 1189-1232, 2001.

[20] C. &. V. V. Cortes, "Support Vector Networks. Machine Learning", 1995.

[21] T. &. H. P. Cover, "Nearest Neighbor Pattern Classification. IEEE Transactions on Information Theory", 1967.

[22] Q. Du, T. Xie and Y. He, "Anomaly Detection and Diagnosis for Container-based Microservices with Performance Monitoring", *EasyChair Preprint*, Vol. 468, 2018, doi: 10.48550/ezcheir.2018.468.

[23] Y. Lyu, H. Li, M. Sayagh, Z. Jiang and A. E. Hassan, "An Empirical Study of the Impact of Data Splitting Decisions on the Performance of AIOps Solutions", *ACM Transactions on Software Engineering and Methodology*, Vol. 30, No. 4, pp. 1-38, 2021, doi: 10.1145/3447.

[24] C. Zhao, M. Ma, Z. Zhong, Z. Tan, L. Yu, Y. Sun, D. Pei and Q. Lin, "Robust multimodal failure detection for microservice systems", In: *Proc. of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5639-5649, 2023, doi: 10.1145/3580305.3599902.

[25] L. M. A. QASSEM, T. STOURAITIS, E. DAMIANI and I. (. M. ELFADEL, "Proactive Random-Forest Autoscaler for Microservice Resource Allocation", *arXiv*, 2021, doi: 10.48550/arXiv.2105.02863.

[26] A. Bento, J. Correia, R. Filipe, F. Araujo and J. Cardoso, "Automated Analysis of Distributed Tracing: Challenges and Research Directions", *Journal of Grid Computing*, Vol. 19, 2021, doi: 10.1007/s10723-021-09551-5.

[27] Q. Lin, K. Hsieh, Y. Dang, H. Zhang, K. Sui, Y. Xu, J.-G. Lou, C. Li, Y. Wu, R. Yao, M. Chintalapati and D. Zhang, "Predicting Node Failure in Cloud Service Systems", *ESEC/FSE '18*, Lake Buena Vista, FL, USA, 2018.

[28] I. Tarhri, "k8s-failure-dataset-generator: A collection of code and dataset for Kubernetes failure prediction", 2024, [Online]. Available: https://github.com/ilyasstrh/k8s-failure-dataset-generator

[29] J. Chen, H. Huang and H. Chen, "Informer: Irregular Traffic Detection for Containerized Microservices RPC in the Real World", In: *Proc. of the 4th ACM/IEEE Symposium on Edge Computing (SEC '19)*, pp. 389–394, 2019, doi: 10.1145/3318216.3363375.

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/